Nakijk ERD — V1

NAKIJK V1 · Databaseproject · ERD (relationeel modelleren) · bij de Projectopgave Databases WT · 22 oktober 2026

Nagekeken: `Project Database/1_Project_ERD/Bouwcontrole ERD.drawio` (22 oktober, 22:13).

> **OPMERKING**
>
> Dit is een nakijk van je eerste aanzet: negen entiteiten met een primaire sleutel, één alternatieve sleutel, een enumeratie en een notitie voor de soorten en de relatiebeschrijvingen. Relaties, attributen, datatypes, NA/NNA en delete-opties staan er nog niet in. Die kijk ik na in een volgende versie.
>
> Deze nakijk volgt de regel van je leerkracht: **enkel wijzen naar wat juist of fout is, geen oplossingen.** Daarom staat er bij de bevindingen geen *Richting*, zoals bij Programming Advanced. Je krijgt per bevinding waar het zit, wat er misloopt, waarom het telt, en hoe je het zelf controleert.
>
> Achteraan staat je ERD, met bij elke bevinding een label op de plaats waar ze zit.
>
> De maatstaf: de projectopgave, de leerstof van les 1, 2, 3 en 12 (je leerboek Database, deel 1), en je eigen Bouwcontrole-documenten.
>
> Deze nakijk staat in `Hulpmiddelen\Nakijk`, buiten `Project Database`. Zo komt hij nooit in de ZIP die je inlevert.

In één oogopslag

Je model staat aan het begin, en het begin is goed gekozen: je begint bij de entiteiten, de eerste stap van de synthesemethode. Wat er staat, volgt grotendeels de regels van de cursus.

Vier dingen gaan tegen een regel in (A1 tot A4), twee dingen spreken elkaar tegen (B1 en B2), en er zijn vijf opmerkingen voor later (C1 tot C5). De belangrijkste zijn **A1 en A2**: ze gaan over de soort van drie entiteiten, en daardoor kloppen drie tellingen uit de opgave nog niet.

Wat er goed zit

Wat	Waarom het goed is	Waar het staat
Je begint met de entiteiten	De synthesemethode begint bij de vraag waarover je gegevens bijhoudt. Wat je erover bijhoudt, komt pas daarna. Buiten de sleutels heb je nog geen attributen ingevuld.	hoofdstuk 2, *De synthesemethode*
Elke entiteit heeft een primaire sleutel, bovenaan	Elke tabel heeft een PK nodig om haar rijen uit elkaar te houden. Door ze overal meteen te zetten, vergeet je er geen. Hoe ze heet, staat bij A4.	hoofdstuk 1 en 5
`Onderdeel` heeft een alternatieve sleutel met de artikelcode	Precies wat de les vraagt: een natuurlijke sleutel uit de werkelijkheid wordt een AK, en de PK blijft een surrogaatsleutel zonder betekenis. Verandert een code ooit, dan breekt er niets wat ernaar verwijst.	hoofdstuk 5, *Alternatieve sleutels*
Je denkt vanaf het begin aan de soort van elke entiteit	De opgave vraagt de benoeming van de soort bij elke entiteit (*Vereisten*). Wie dat pas op het einde doet, ontdekt laat dat een telling niet klopt.	opgave, *Vereisten*; hoofdstuk 8
`OnderdeelType` met de vorm *List*, met de waarden eronder	Zo tekent de cursus een enumeratie in draw.io. Wat er tegenstrijdig aan is, staat bij A2.	hoofdstuk 9, *Zelfgemaakte types*

Wat	Waarom het goed is	Waar het staat
`Klantcontact` bij de associatie-entiteiten	De naam zegt welke twee entiteiten ze koppelt: `Klant` en `Contact`, en die staan allebei in je model. Dat past bij de definitie. Of het echt een veel-op-veelrelatie is, blijkt pas als je de relaties tekent.	hoofdstuk 6 en 8
De entiteiten staan, op één na, in het enkelvoud en met een hoofdletter	Dat is de regel uit de ERD-checklist van les 1. De uitzondering staat bij A3.	hoofdstuk 4
Je werkt in een `.drawio`-bestand	De opgave aanvaardt enkel het `.drawio`-bestand, geen `.png` of `.jpeg`.	opgave, *Upload*
Je bron is groot genoeg	De opgave raadt een onderwerp aan dat groter is dan wat je moet bouwen. De werkvloer van Alternate geeft je meer dan genoeg om uit te kiezen.	opgave, *Onderwerp*

De eisen van de opgave

Eis uit de opgave	Wat er nu staat	Status
minstens 10 entiteiten	9 entiteiten, plus de lijst `OnderdeelType`	**½ 9 of 10, zie A2**
3 kernentiteiten	de lijst in je notitie is nog leeg	nog te doen
2 associatie-entiteiten	3 genoteerd; bij 2 ervan zijn geen twee entiteiten aan te wijzen die ze koppelen	**✗ zie A1**
1 karakteristieke entiteit	`OnderdeelType` genoteerd, maar getekend als enumeratie	**✗ zie A2**
4 vrij te kiezen entiteiten	hangt af van de soorten hierboven	nog niet te beoordelen
één kernentiteit met minstens 8 attributen, de andere minstens 4	nog geen attributen	nog te doen
minstens één special topic	nog geen	nog te doen
minstens één enumeratie	`OnderdeelType`, zolang het een enumeratie is	**½ zie A2**
relaties met multipliciteit	nog geen	nog te doen
relatiebeschrijvingen	zeven lege plaatsen in je notitie	nog te doen, zie C4
primaire en foreign keys	een PK bij elke entiteit, nog geen FK	**½ zie A4**
datatypes en NA/NNA	nog niet	nog te doen
DTR, DTN of DTC	nog niet	nog te doen
eventuele alternatieve sleutels	bij `Onderdeel`. De ERD-checklist van les 2 vraagt er één bij elke kernentiteit	nog niet te beoordelen: de kernentiteiten zijn nog niet benoemd
benoeming van de soort	in de notitie, voor 4 van de 10	**½ zie A1, A2 en B1**

A. Wat tegen een regel of eis ingaat

A1. `ContactSoort` en `KlantType` staan bij de associatie-entiteiten

Waar: de notitie rechtsonder, onder *associatie-entiteiten*, en de entiteiten `Contactsoort` en `Klanttype`.

Wat een associatie-entiteit is. Ze stelt een veel-op-veelrelatie voor tussen twee entiteiten. Ze bestaat om die twee aan elkaar te koppelen, en heeft een foreign key naar elk van beide. De eerste vraag uit de tabel *Hoe je de soort herkent* in hoofdstuk 8 is: *heeft ze twee foreign keys, en bestaat ze om twee andere entiteiten aan elkaar te koppelen?*

Waarom deze twee niet voldoen, zoals ze nu staan. Bij `Klantcontact` kan je de twee entiteiten die ze koppelt al aanwijzen in je model: `Klant` en `Contact`. Bij `ContactSoort` en `KlantType` niet. Het tweede deel van de naam is geen entiteit in je model: er is geen `Soort` en geen `Type`. Er zijn geen relaties getekend, en ook geen andere twee entiteiten waartussen ze een veel-op-veelrelatie voorstellen.

Hoofdstuk 8 waarschuwt dat een associatie-entiteit soms een eigen naam heeft, zoals `Beoordeling`, die niet naar de twee entiteiten verwijst. Daarom gaat het niet om de naam, maar om de vraag of je die twee entiteiten kan aanwijzen. Die vraag staat hieronder.

Waarom het telt.

- De opgave vraagt de benoeming van de soort bij elke entiteit (*Vereisten*), en heeft een eigen feedbackcriterium *Benoemen entiteiten*.
- Kan je bij deze twee geen twee entiteiten aanwijzen, dan blijft er één associatie-entiteit over. De opgave vraagt er twee.
- De telling van de andere soorten verandert mee.

Zo controleer je het zelf.

- Stel bij elke entiteit de drie vragen uit *Hoe je de soort herkent* (hoofdstuk 8), in die volgorde. De eerste vraag waarop je ja antwoordt, geeft de soort.
- Schrijf bij elke associatie-entiteit de zin die de veel-op-veelrelatie beschrijft, in beide richtingen: *een … kan bij meerdere … horen, en een … kan bij meerdere … horen*. Lukt die zin niet met twee entiteiten uit je model, dan is het geen associatie-entiteit.

A2. `OnderdeelType` is getekend als enumeratie, maar benoemd als karakteristieke entiteit

Waar: de lijst `OnderdeelType`, midden onderaan, en de notitie onder *karakteristieke entiteit*.

Wat de tekening zegt. Je tekende `OnderdeelType` met de vorm *List*, met de waarden eronder en zonder primaire sleutel. Zo tekent de cursus een **enumeratie**: een vaste lijst waarden in het model zelf, zonder eigen entiteit en zonder foreign key.

Wat de notitie zegt. Daar staat ze als **karakteristieke entiteit**. Dat is een entiteit op zich, met een eigen primaire sleutel.

Allebei tegelijk kan niet. Wie je model beoordeelt, kan niet weten welke van de twee je bedoelt.

Waarom het telt.

- In de verdeling van de entiteiten vraagt de opgave één karakteristieke entiteit. Daarnaast vraagt ze, apart, minstens één enumeratie. Eén element kan niet voor allebei tellen.
- Een enumeratie is geen entiteit. Of je model nu 9 of 10 entiteiten telt, hangt ervan af wat `OnderdeelType` is.

Zo controleer je het zelf.

- Lees *Zelfgemaakte types* en *Hoe je kiest* in hoofdstuk 9. Daar staan de twee vormen naast elkaar, met de vuistregel uit les 3.
- Kijk daarna of je tekening en je notitie hetzelfde zeggen.

A3. `Conflicten` staat in het meervoud

Waar: de entiteit linksonder.

Wat. Een entiteit staat in het enkelvoud. De naam zegt wat één rij in de tabel voorstelt, niet wat de hele tabel bevat. Al je andere entiteiten volgen die regel.

Waarom het telt. Het staat in de ERD-checklist van les 1: *elke entiteit begint met een hoofdletter en staat in het enkelvoud*. De naam van een entiteit wordt later ook de naam van de tabel.

Zo controleer je het zelf. Loop de tabel *De regels* in hoofdstuk 4 af voor elke naam in je model, ook voor de namen in je notitie.

A4. De primaire sleutel en de artikelcode beginnen met een hoofdletter

Waar: de eerste rij van elke entiteit (`Id`, negen keer), en de tweede rij van `Onderdeel` (`Artikelcode`).

Wat. In de cursus heet de primaire sleutel altijd `id`, met een kleine letter. Elk attribuut begint met een kleine letter, en een naam van meer woorden schrijf je in camelCase.

Waarom je het zo deed, vermoedelijk. In C# begint een eigenschap met een hoofdletter. In je klasse `Onderdeel` van Bouwcontrole heet het ook `Id` en `KrakenId`, en daar is dat juist. Het ERD volgt een andere afspraak: die van de cursus Databases. Twee vakken, twee afspraken.

Waarom het telt. De ERD-checklist van les 1 zegt: *elk attribuut begint met een kleine letter*. Een PK is ook een attribuut.

Zo controleer je het zelf. De tabel *De regels* in hoofdstuk 4, de rijen *attribuut* en *primaire sleutel*.

B. Wat elkaar tegenspreekt

B1. De namen in je notitie wijken af van de namen in je tekening

Waar: de notitie, onder *associatie-entiteiten*, en de entiteiten zelf.

Wat.

In de tekening	In de notitie
`Contactsoort`	`ContactSoort`
`Klanttype`	`KlantType`
`Klantcontact`	`Klantcontact` ✓
`OnderdeelType`	`OnderdeelType` ✓

Ook in de tekening zelf schrijf je hetzelfde patroon, een entiteit met *soort* of *type* erachter, op twee manieren: `OnderdeelType` met een hoofdletter in het midden, `Klanttype` en `Contactsoort` zonder.

Waarom het telt. Naamgeving moet consequent gebeuren (hoofdstuk 4, *Eén taal, overal*). Wie je model beoordeelt, legt je notitie naast je tekening. Twee schrijfwijzen lezen als twee verschillende dingen.

Zo controleer je het zelf. Leg de notitie naast de tekening en vergelijk elke naam letter per letter, ook de hoofdletters.

B2. Je lijst `OnderdeelType` wijkt af van je eigen Bouwcontrole-documenten

Waar: de waarden van `OnderdeelType`.

Wat. Je C#-modellen in Bouwcontrole en je document *Eigenschappen per onderdeeltype* tellen elk tien onderdeeltypes. Je lijst telt er ook tien, maar niet dezelfde tien:

Wat	In je C#-modellen	In je eigenschappen-document	In je lijst
een koeler	`ProcessorKoeler`	koeler	staat er niet in
een besturingssysteem	staat er niet in	staat er niet in	besturingssysteem

De andere negen zitten in alle drie, soms onder een andere naam, zoals processor en CPU. Ik vergeleek met je C#-modellen van 14 oktober en met de versie van je eigenschappen-document van 20 september. Heb je ze sindsdien aangepast, leg dan je eigen versie ernaast.

Waarom het telt. De opgave vraagt een ERD dat de bron zo getrouw mogelijk nabouwt. Je bron is de werkvloer van Alternate, en je Bouwcontrole-documenten beschrijven die. Een verschil tussen je model en je documenten kan bewust zijn, of niet. Dat kan ik uit je model niet afleiden. Aandachtspunt A04 van je takenlijst gaat hierover: de twee maatstaven.

De opgave raadt aan om elke keuze die verwarring kan geven, te verantwoorden in een README of Word-document (*Verduidelijkingen*).

Zo controleer je het zelf. Leg je lijst naast de map Models van Bouwcontrole, en naast punt 4 van *Wat ik graag terug hoor*, achteraan in je eigenschappen-document.

C. Opmerkingen

Geen fouten, wel goed om nu al te weten.

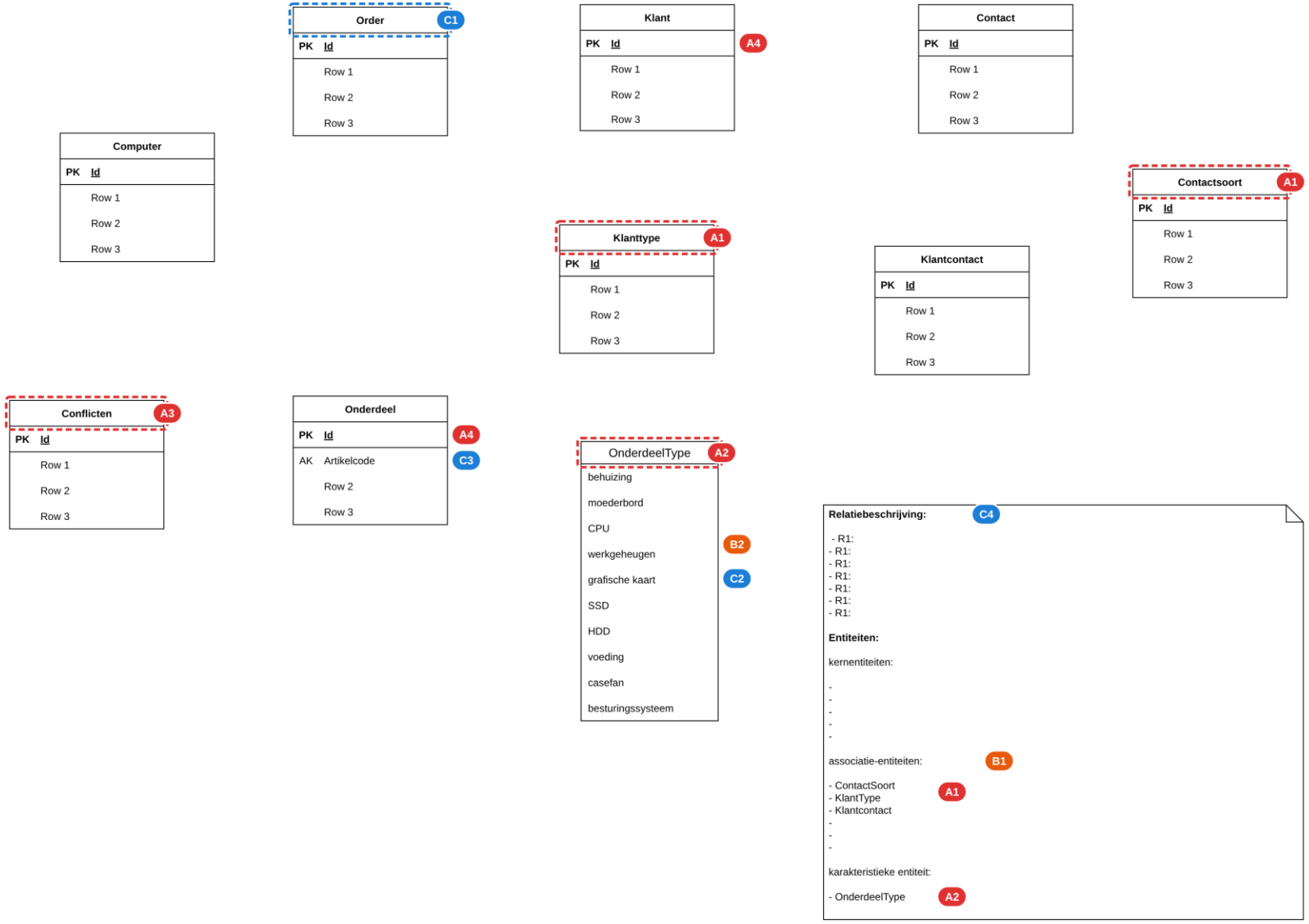
Nr.	Waar	Wat	Waarom het goed is om te weten
C1	`Order`	ORDER is een gereserveerd woord in SQL Server, zoals in ORDER BY. Voor het ERD is het geen fout: de cursus gebruikt zelf `Order`, in het voorbeeld van Harry en Saartje.	SQL Server aanvaardt een gereserveerd woord enkel als naam met een speciale schrijfwijze (Microsoft, *Reserved Keywords*). Je botst er dus op in het SQL-deel, niet nu.
C2	de waarden van `OnderdeelType`	Ze zijn niet op één manier geschreven: `CPU`, `SSD` en `HDD` in hoofdletters, de andere in kleine letters, en `grafische kaart` met een spatie.	De cursus legt geen regel op voor de waarden zelf, wel dat je consequent bent (hoofdstuk 4). Een vaste lijst bestaat net om een waarde overal op dezelfde manier te schrijven (hoofdstuk 9).
C3	`Artikelcode` in `Onderdeel`	Een alternatieve sleutel moet echt uniek zijn. Twee rijen mogen nooit dezelfde waarde hebben.	Hoofdstuk 5, *Veelgemaakte fouten*, noemt een AK die niet echt uniek is. Ga voor jezelf na of een artikelcode bij Alternate nooit twee keer voorkomt.
C4	de notitie, *Relatiebeschrijving*	Zeven plaatsen, die allemaal `R1` heten.	Elke relatie krijgt een eigen volgnummer, en de FK van die relatie krijgt hetzelfde nummer (hoofdstuk 4 en 6).
C5	de map `1_Project_ERD`	Naast je ERD staat `.$Bouwcontrole ERD.drawio.bkp`. draw.io maakt dat back-upbestand zelf aan.	Het hoort niet bij wat je inlevert: de opgave vraagt het `.drawio`-bestand. Zonder `.gitignore` gaat het wel mee naar GitHub als je de map pusht.

Wat ik nog niet kon beoordelen

Dit staat nog niet in je model. Ik kijk het na in een volgende versie, en dan lopen ook de zeven feedbackcriteria van de opgave volledig mee.

- de relaties: de plaats van de foreign keys, de veel-op-veelrelaties, de multipliciteiten en de relatiebeschrijvingen
- de attributen en hun aantallen, de datatypes, NA en NNA
- de delete-opties
- het special topic
- wat één rij van `Contact`, `Contactsoort`, `Computer` en `Conflicten` voorstelt. Zonder attributen of relaties kan een lezer dat nog niet afleiden. Kan hij het later uit je model niet halen, dan raadt de opgave een verantwoording aan in je README (*Verduidelijkingen*).

Bijlage: je ERD, met de bevindingen erop



Je ERD zoals het nu is, nagetekend uit je .drawio-bestand. Er is niets aan veranderd. De labels tonen waar elke bevinding zit: rood is A (tegen een regel), oranje is B (tegenstrijdig), blauw is C (opmerking).